except for a default price service, which returns `"XX"`. The US version is shown in listing 7.13. Others are available in the download that accompanies this section (http://mng.bz/o54p).

Listing 7.13 Example implementation for an `IPriceService` for the USA

```
public class UsPriceService : IPriceService
{
    public string GetLocation() => "us";
    public double CalculatePrice()
    {
        ...
    }
}
```

You register a variety of implementations with the service container:

```
builder.Services.AddScoped<IPriceService, FrPriceService>();
builder.Services.AddScoped<IPriceService, GbPriceService>();
builder.Services.AddScoped<IPriceService, UsPriceService>();
builder.Services.AddScoped<IPriceService, DefaultPriceService>();
```

If you were to inject `IPriceService` into a `PageModel` constructor, you would always get the `DefaultPriceService`, as established above, being that it is the last one registered. However, you can also inject an `IEnumerable<IPriceService>`, which resolves to a collection of all registered implementations. Then it is just a question of selecting the implementation that applies to the current request.

I'm a fan of Cloudflare (https://www.cloudflare.com/), which provides a range of web-related services, including geolocation (other geolocation service providers are available), whereby they identify the location of a request based on its IP address. The location is made available to application code in the request headers as an ISO-3166-1 Alpha-2 code, or `"XX"` where it is not possible to resolve the location. The following listing shows an